

# PYTHON PROGRAMMING

## Setting up the environment

-This course assumes that the student uses the Pycharm IDE.

-For windows and MacOS users, visit [www.jetbrains.com](http://www.jetbrains.com) and download the respective installer file (.exe for windows and .dmg for MacOS)

-For linux users(debian based), go to corresponding app store and search for Pycharm IDE either in .deb, Snap or Flatpak.

-For non Debian -based linux systems, visit [www.jetbrains.com](http://www.jetbrains.com) and download the linux tarball package(.tar.gz)

Once installed, launch Pycharm and follow the instructions on the screen. Download/Select an interpreter (Python 3.14 at the time of writing of this document).

## Introduction

Computers do not understand human language, so we write a program in a language which a computer can understand. There are many different programming languages like Kotlin, Rust, C, C++, Java, Perl, Ruby, Python, etc. In this course, you will learn about Python.

Guido Van Rossum created Python while he was working at CWI (Centrum Wiskunde & Informatica), which is based in the Netherlands. Python was released in 1991. Python got its name from a BBC comedy series from the seventies- "Monty Python's Flying Circus".

Some of the features which make Python different from other programming languages are as follows:

### 1. Easy to code and easy to read

Python is very easy to code as compared to other popular languages like Java and C++. It is straightforward for anyone to learn Python syntax in just a few hours. Also, Python is easy to read as it is similar to the English language.

### 2. Free and Open-Source

Python is freely available. You can download it from <https://www.python.org/downloads/>

Also, it is open-source. It means its source code is open to the public. One can download it, change it, use it, and distribute it without any restrictions.

### 3. High-Level

It is a high-level language, meaning one does not need to remember the system architecture. Also, as a programmer, we do not need to manage memory. These features make Python more programmer-friendly.

### 4. Portable

Suppose you write a Python code for Windows and want to run on Mac. Then you do not need to make changes to it. In simple words, you can run Python code on any machine. You don't need to write different codes for different machines. This feature makes Python a portable language.

## 5. Interpreted

If you have written code in C++ or Java, you must first compile it and then run it. But in the case of Python, you don't need to compile it. By interpreted, we mean the source code is executed line by line, and not all at once.

## 6. Object-Oriented

An object-oriented language is a language that can model the real world. On the other hand, a procedure-oriented language focuses on functions, which are code that can be reused. Python supports both object-oriented and procedure-oriented programming.

## 7. Large Standard Library

Python has an extensive library, meaning you don't have to write code for every single thing. The library comes with pre-written codes which you can use while writing codes. There are libraries for regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, image manipulation, and many more.

To code an actual program, you will have to write your code in a text file and save it with a .py extension.

This type of file is also known as a Python script.

To create a Python script, simply click on File > New File on the menu bar at the top of your Python Shell. This will open a new window in the form of text editor where you will write your program.

Let us write our first program called the "Hello World" program.

Type the following code into your text editor.

```
#This code prints the words "Hello World"

print("Hello World")
```

As shown in the figure below, if you closely notice, line `#This code prints the words "Hello World "` is in red. On the other hand, the word `'print'` is in purple, and `"Hello World "` is in green. The words `print` and `"Hello World "` serve different purposes in our program, so they are displayed in different colors.

In the above code `#This code prints the words "Hello World"` which is displayed in red is not part of the program. This is a comment written to make our code more readable for other programmers. The Python interpreter ignores this line of code. To add comments to your program, simply type a `#` sign before writing a comment as typed below:

```
#This is a comment
```

```
#This is also a comment
```

```
#This is yet another comment
```

Also, we can use three single quotes for writing multiline comments, for example:

```
'''
```

```
This is a comment
```

```
This is also a comment
```

```
'''
```

To save your code, simply click on File > Save As... keep in mind to save your code with .py extension.

Finally, to execute the program, simply click on Run > Run Module or press F5. Now you will see the words Hello World printed on your Python Shell.

## Variables and Operators

In this chapter, you will learn about variables and operators. You will learn about what variables are and how to declare them and name them. You will also learn about the common operations that you can perform on variables.

### **What are Variables?**

A variable is a memory location which is used to store data values. This stored value is processed by the computer and is used to manipulate data.

For example, let us say that your program needs to store the name of the user. To do that, you need to name this data username and assign it an initial value. In Python assigning an initial value will define the variable username. We define the userName using the following statement.

```
userName = 'Peter'
```

Once you define the variable userName , your program will allocate a fixed area of your computer storage space to store this data. Keep in mind that every time you declare a new variable, you need to assign an initial value to it. In the above example, we have assigned the value Peter; this value can be changed in your program later.

You can also define multiple variables at one time, as shown below:

```
userAge, userName = 69, 'Peter'
```

You can also write the above statement, as shown below:

```
userAge = 69
```

```
userName = 'Peter'
```

### **How to Name a Variable?**

In Python, a variable name can only contain letters (a-z, A-B), numbers, and underscores (\_).

The first character of a variable cannot be a number. Some of the valid variables are userName , user\_name , user2Name. The invalid variable example will be 10userName .

Variable names are case sensitive, meaning username is not the same as userName. You cannot use reserved keywords to name a variable; some of the reserved keywords are print, if, while, input, etc.

## Assignment Operator

An assignment operator is used to assign a particular value to a variable.

The sign = is known as an assignment operator. The value on the right-hand side of the = sign gets assigned to the variable on the left. As we have seen in the above example, userAge = 69, we have assigned 69 to variable userAge .

Let us type the following code in IDE editor and save it.

```
x = 15
```

```
y = 101
```

```
x = y
print ("x = ", x)
print ("y = ", y)
```

Once you run the program you will get the following output:

```
x = 101
y = 101
```

Initially, 15 is assigned to x variable on the first line, 101 is assigned to y variable in the next line value of y is assigned to x .

Now let's change the program by changing the third line from `x = y` to `y = x` . In mathematics, `x = y` and `y = x` mean the same thing. But not in programming.

Once you run the second program, you will get the following output:

```
x = 15
y = 15
```

As we can see from above, the value of x remains as 15, on the other hand the value of y is changed to 15. This is because the statement `y = x` assigns the value of x to y.

## Basic Operators

Basic operators in Python are `+`, `-`, `*`, `/`, `//`, `%`, and `**`, which are addition,

subtraction, multiplication, division, floor division, modulus, and exponent. Using these signs, we can perform the mathematical operations on variables.

Examples are:

Let us assume **x = 20, y = 10**

Addition:

```
x + y = 30
```

Subtraction:

```
x - y = 10
```

Multiplication:

```
x*y = 200
```

Division:

```
x/y = 2
```

Floor Division:

```
x//y = 2 (this operator is used to round down the answer to the nearest whole number)
```

`x%y = 0` (it returns the remainder when 20 is divided by 10)

Exponent:

$x^{**}y = 10240000000000$  (20 to the power of 10)

## More Assignment Operators

Apart from basic assignment operators, there are few more assignment operators in Python. These include operators like `+=`, `-=` and `*=`.

Let us suppose we have the variable `x`, with an initial value of 20, then the statement `x = x + 10` can be written as `x += 10`.

The above statement states that the expression on the right-hand side, i.e, `(x+ 10)` is first evaluated and the answer gets assigned to the left. So the above statement becomes `x = 30`.

Hence we conclude that `x = x + 10` can be written as `x +=10` which expresses the same meaning.

## Data Types

A data type is the type of data that a variable stores. In this chapter, we will look into some basic data types in Python like **integers**, **float** and **string**.

After that we will discuss about the concept called type casting. And finally we will discuss about more data types in Python: the list, tuple and dictionary.

### Integers

Integers are data types that are numbers that have no decimal parts, for example, -5, -4, -3, 0, 5, 7, etc.

To declare an integer in Python, simply write `varName = initial value`

For instance:

```
ageofUser = 32
```

```
pinCode = 347743
```

### Float

Float are numbers that have decimal parts such as 1.234, -0.04342, 13.56. To declare a float in Python, we will write `varName = initial value`

For instance:

```
height = 2.56
```

```
weight = 76.2
```

### String

Strings are text.

You can declare strings in the following ways:`varName = 'initial value'` (single quotes) or

`varName = "initial value"` (double quotes)

Examples are:

```
name = 'Akash'
```

```
siblingsName = "Aron"
```

```
age = '40'
```

In the last example, we wrote `age = '40'`, which means `age` is a string (because we have defined `age` with single quotes); however if we write `age= 40`, then `age` is an integer because single quotes or double quotes is not included.

You can combine multiple substrings by using the concatenate sign (+). For example, `"Aron" + "James"` is equivalent to `"Aron James"`.

## Type Casting

Sometimes we need to convert from one data type to another. This is known as type casting.

To perform type casting, Python has three built-in functions (we will learn about functions in a later chapter). These are **int()**, **float()**, and **str()** functions.

The `int()` function takes in a float or a string and converts it to an integer.

To change a float to an integer, simply type `int(6.764543)`. We will get 6 as a result as you can see anything after the decimal point is removed. To change a string to an integer, we will type `int("4")`, and we will get 4 as an output.

The `float()` function takes an integer or a string and converts it to a float. For example, if we type `float(5)` or `float("5")`, we will get 5.0 as an output. If we type `float("5.72486")` we will get 5.72486 as an output. As you can see, the quotations marks are removed; hence it is converted into a float. The `str()` function converts an integer or float to a string. For example, if we type `str(5.2)`, we will get "5.2" as output.

## List

Lists are a collection of data that are related to each other. Suppose we need to store names of 5 users; instead of storing names in 5 variables we can store names in one variable with the help of lists. Here is how you will declare a list, simply write:

```
listName = [value1, value2, value3, value4, value5] keep in mind that we
```

use square brackets `[]` while declaring a list. A comma separates multiple values.

Example:

```
userName = ['Peter', 'John', 'Emile', 'Jackson', 'Anna']
```

As you can see in the above example, we have declared a list variable `userName` and have assigned the names to the variable.

To declare a list without assigning an initial value simply write `listName = []`.

Each value in the list is assigned an index number; indexes always start from ZERO. Hence the first value has an index of 0; the next has an index of 1, and so on. For example, `userName[0] = 'Peter'`, `userName[1] = 'John'`

You can also assign a list to a variable. So if we write `userName1 = userName`, the variable `userName1` will become `['Peter', 'John', 'Emile', 'Jackson', 'Anna']`.

## Tuple

Tuples are similar to lists, but you cannot modify their values once you declare tuples and assign initial values to it that value will stay for the rest of the program. To declare a tuple, simply write `tupleName = (initial values)`. Keep in mind that parentheses `()` are used while declaring a tuple. And the multiple values are separated by a comma.

Example:

```
daysOfWeek = ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")
```

To access the individual values of tuple, we will be using their indexes similar to list. For instance, `daysOfWeek[0] = "Monday"`, `daysOfWeek[2] = "Wednesday"`.

## Dictionary

Dictionary is an unordered collection of items. Each item of a dictionary has a **key/value** pair. To declare a dictionary we simply write

`dictionaryName = {dictionary key : data}`. We use curly braces `{}` while declaring a dictionary, and commas separate the multiple pairs.

Example:

```
userNameAndHeight = {"John" : 4.2, "Aron" : 5.3, "Alvin" : 6.5}
```

We can also declare a dictionary using the `dict ()` function. We can declare the `userNameAndHeight` dictionary above by writing:

```
userNameAndHeight = dict (John = 4.2, Aron = 5.3, Alvin = 6.5)
```

When you use `dict ()` function to declare a dictionary we use parentheses `()` in place of curly braces `{}` and you do not put quotation marks for the dictionary keys.

To access individual items in the dictionary, we use the dictionary key. For example, to get John's height, you will write `userNameAndHeight ["John"]`.

And you will get the value 4.2.

To modify items in a dictionary, we simply write `dictionaryName[dictionary key of item to be modified] = new data`.

For example, to modify the `"Aron" : 5.3` pair we will write `userNameAndHeight ["Aron"] = 5.1`. Now our dictionary becomes `userNameAndHeight = {"John" : 4.2, "Aron" : 5.3, "Alvin" : 6.5}`. Chapter 5-Uses of `input ()` and `print ()` Functions In this chapter, we will know how we can make our programs more interactive with the use of `input ()` and `print ()` functions.

Let us say that we want to write a program that displays your name and age on the screen. To do that, we need to use two functions called `input ()` and `print ()`.

Simply type the following code:

```
name = input ("Enter the name please: ")
age = input ("What is your age: ")
print ("My name is ", name, "and I am", age, "years old.")
```

Once you run the program, you will see the following output on the screen.

*Enter the name please:*

Here you need to enter a name, suppose you entered Alex. Now press Enter, and you will see the following output on the screen.

*What is your age:*

Now suppose you entered 30. After that, press Enter again. Now you will get the following output on the screen.

My name is Alex, and I am 30 years old.

## input()

As you can see in the above example, we have used the input() function two times to get out the user's name and age.

```
name = input ("Enter the name please: ")
```

The above statement instructs the user to enter the name. Once the statement "Enter the name please:" gets printed on the screen. The input() function waits for the user to enter the name. Once the name is entered by the user, the information gets stored as a string in the variable name. Now the next input() function gets executed. The following statement gets executed.

```
age = input ("What is your age: ")
```

Now "What is your age:" is gets printed on the screen, asking the user to enter the age. Once the user enters the age, it gets stored as a string in the variable age. This is how the input() function works.

## print()

The print() function is used to display a particular information to users. As in the statement below, we can see the print() function has 5 arguments.

```
print ("My name is ", name, "and I am", age, "years old.")
```

The first argument is the string "My name is" the next is the variable name which was declared using the input() function. Next is the string, "and I am" next is the variable age, and the final argument is "years old."

## Triple Quotes

Triple quotes (''' or ''') are used to display multiple lines using the print() function. For example,

```
Print ('''My name is Alex and I am 30 years old.''' )
```

Above statement will give us the following output:

My name is Alex

and I am 30 years old.

In this manner we can increase the readability of our message.

## Escape Characters

In some cases, we need to print some special characters such as a tab or a newline. To do this, we will use the \ (backlash) character.

For example, if we want to print a tab, then we will type the backslash character before the letter t. for instance, if you write `print ('Hello\tWorld')`, you will get the following output:

```
Hello World
```

Some other uses of backslash character are given below:

`\n` (This prints a newline)

```
>>> print (Hello\nWorld')
```

```
Hello
```

```
World
```

`\\` (This will print the backslash character itself)

```
>>> print ('\\')
```

```
\
```

`\"` (This will print the double quotes)

```
>>> print ("My height is 6'7\" tall")
```

```
My height is 6'7" tall
```

`'` (This will print the single quote)

```
>>> print ('My height is 6\'7" tall')
```

```
My height is 6'7" tall
```

## Conditional Statements

In this chapter, we will learn about the **if statement**, **for loop**, and **while loop**. But before we dive into **for**, **if**, and **while** loop statements, let us understand what a conditional statement is.

### Condition Statements

A conditional statement is a statement that compares two variables. Suppose we want to compare whether two variables are the same or not. To do this, we will use the `==` operator (double `=`). For example, if you want to check whether the value of `x` is equal to the value of `y`, we will write `x==y`. if the condition is met we will get `True`, or else we will get `False`.

Some other comparison operators are `!=` (not equal), `<` (smaller than), `>`

(greater than), `<=` (smaller than or equal to) and `>=` (greater than or equal

to). For instance,

Not equal:

```
4 != 5
```

Greater than:

```
7>5
```

Smaller than:

8<9

Greater than or equal to:

7 >= 4

6 >= 6

Smaller than or equal to:

1 <= 7

5 <= 5

The above statements will be evaluated to True.

## if Statement

If statement is a decision-making statement that is used in Python. The syntax of if statement is as follows:

```
if condition 1 is met :
```

```
do A
```

```
elif condition 2 is met :
```

```
do B
```

```
elif condition 3 is met :
```

```
do C
```

```
elif condition 4 is met :
```

```
do D
```

```
else:
```

```
do E
```

In the above syntax elif stands for “else if” you can have as many elif statements as you wish.

Now let’s write a program to understand if statement better:

```
num = 4
```

```
if num > 0:
```

```
print("Positive number")
```

```
elif num == 0:
```

```
print("Zero")
```

```
else:print("Negative number")
```

If you run this program you will get the following output:

Positive number

If num is positive, Positive number is printed. If num is equal to 0, Zero is printed. However, if num is negative, Negative number is printed.

In the above code we have assigned num = 4 , hence the statement num >0 becomes true so Positive number is printed.

## for Loop

The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. Syntax for for loop is:

```
For iterator_var in sequence:
```

```
statements(s)
```

For example:

```
n=4
```

```
for i in range(0, n):
```

```
print(i)
```

Output:

```
0
```

```
12
```

```
3
```

Another example:

```
Print("List Iteration")
```

```
l = ["geeks", "for", geeks"]
```

```
for i in l:
```

```
print(i)
```

```
print("\nTuple Iteration")
```

```
t = ("geeks", "for", geeks")
```

```
for i in t:
```

```
print(i)
```

```
print("\nString Iteration")
```

```
s = "Geeks"
```

```
for i in s :
```

```
print(i)
```

```
print("\nDictionary Iteration")
```

```
d = dict()
```

```
d['xyz'] = 123
```

```
d['abc'] = 345
```

```
for i in d :  
    print("%s %d" %(i, d[i]))
```

Output:

List Iteration

Geeks

For

Geeks

Tuple Iteration

Geeks

For

Geeks

String Iteration

G

e

e

k

s

Dictionary Iteration

xyz 123

abc 345

We can also include else statement with for loop. As there is no condition in for loop based on which the execution will terminate so the else block will be executed immediately after for block finishes execution. The following program will explain the concept better.

```
list = ["geeks", "for", "geeks"]
```

```
for index in range(len(list)):
```

```
    print list[index]
```

```
else:
```

```
    print "Inside Else Block"
```

Output:

Geeks

For

Geeks

Inside Else Block

## while Loop

while loop executes instructions inside the loop while certain condition remains valid. The syntax of a while statement is:

while condition is true:

do A

For example,

```
count = 7
```

```
while count > 0:
```

```
    print ("Count = ", count)
```

```
    count = count - 1
```

Output:

```
Count = 7
```

```
Count = 6
```

```
Count = 5
```

```
Count = 4
```

```
Count = 3
```

```
Count = 2
```

```
Count = 1
```

## Break

In loop statement sometimes we need to exit the loop when a particular condition is met. In order to do that we will use break keyword. For instance run the following program,

```
k=0
```

```
for i in range(5):
```

```
    k=k+2
```

```
    print ('i = ', i, ', k = ', k)
```

```
    if k == 6:
```

```
        break
```

Output:

```
i = 0, k = 2i = 1, k = 4
```

```
i =2, k = 6
```

## Continue

continue keyword is used in loop statements. Once we use continue statement, the rest of the loop after the keyword is skipped for that iteration.

For example,

```
j=0
for i in range(5):
    j=j+2
    print ('\ni = ', i, ', j = ', j)
    if j == 6:
        continue
    print ('I will be skipped over if j=6')
```

Output:

```
i=0,j=2
I will be skipped over if j=6
i=1,j=4
I will be skipped over if j=6
i=2,j=6
i=3,j=8
I will be skipped over if j=6
i = 4 , j = 10I will be skipped over if j =6
```

## Functions

Functions are pre written codes which perform a specific task. We use def keyword to define functions.

Here is an example:

```
def my_function():
    print("Hello this is a function")
```

To call a function we use the function name followed by parenthesis:

Example:

```
def my_function():
    print("Hello this is a function")
my_function()
```

Output:

```
Hello this is a function
```